National University of Computer and Emerging Sciences

Assignment No. 1

Generative AI

AI4009

Spring 2026

Submission Instructions:

- Submit the complete .ipynb file, which should be named AI_ASS01_XXF_YYYY, where XX represents the batch and YYYY represents the roll number.
- Submit a Word file containing the GitHub repository link, as well as the Medium and LinkedIn post links
- This is a group assignment (max 2 members allowed).
- Any submission not following the submission instructions will not be evaluated.
- **Submit your own work. Accept your current limitations. Grow from your mistakes.**
- **Plagiarism is strictly prohibited and lead to Straight Zero (^_^).**

Question: Neural Storyteller – Image Captioning with Seq2Seq

Objective: Build a multimodal deep learning model that generates natural language descriptions for images using a Sequence-to-Sequence (Seq2Seq) architecture.

1. **Environment Setup**
   - **Platform:** https://www.kaggle.com/
   - **Accelerator:** GPU T4 x2 (Dual GPU)
   - **Dataset:** Add the Flickr30k https://www.kaggle.com/datasets/adityajn105/flickr30k to your notebook.

2. **Part 1: Feature Extraction Pipeline (Mandatory)**

   Task: Since training a CNN alongside an RNN is computationally expensive, we will "cache" the image features first. Use a pre-trained **ResNet50** to convert each image into a **2048-dimensional feature vector**.

   Run the following cell once and it will give you a flickr30k_features.pkl

```
import os, pickle, torch, torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import DataLoader, Dataset
from PIL import Image
from tqdm import tqdm

def find_image_dir():
    # Common Kaggle root
```

```
    base_input = '/kaggle/input'
    # Walk through the input directory to find where the images actually are
    for root, dirs, files in os.walk(base_input):
        # Look for the folder containing a high volume of jpg files
        if len([f for f in files if f.endswith('.jpg')]) > 1000:
            return root
    return None

IMAGE_DIR = find_image_dir()
OUTPUT_FILE = 'flickr30k_features.pkl'

if IMAGE_DIR:
    print(f" Found images at: {IMAGE_DIR}")
else:
    raise FileNotFoundError("Could not find the Flickr30k image directory. Please ensure the dataset
is added to the notebook.")

# --- THE DATASET CLASS ---
class FlickrDataset(Dataset):
    def __init__(self, img_dir, transform):
        self.img_names = [f for f in os.listdir(img_dir) if f.endswith(('.jpg', '.jpeg'))]
        self.transform = transform
        self.img_dir = img_dir

    def __len__(self):
        return len(self.img_names)

    def __getitem__(self, idx):
        name = self.img_names[idx]
        img_path = os.path.join(self.img_dir, name)
        img = Image.open(img_path).convert('RGB')
        return self.transform(img), name

# --- REMAINDER OF THE PIPELINE (AS BEFORE) ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
model = nn.Sequential(*list(model.children())[:-1]) # Feature vector only
model = nn.DataParallel(model).to(device)
model.eval()

transform = transforms.Compose([
    transforms.Resize((224, 224)),
```

```
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

dataset = FlickrDataset(IMAGE_DIR, transform)
loader = DataLoader(dataset, batch_size=128, num_workers=4)
features_dict = {}

with torch.no_grad():
    for imgs, names in tqdm(loader, desc="Extracting Features"):
        feats = model(imgs.to(device)).view(imgs.size(0), -1)
        for i, name in enumerate(names):
            features_dict[name] = feats[i].cpu().numpy()

with open(OUTPUT_FILE, 'wb') as f:
    pickle.dump(features_dict, f)

print(f"Success! {len(features_dict)} images processed and saved to {OUTPUT_FILE}")
```

3. **Part 2: Vocabulary & Text Pre-Processing**

   Task: Load the captions.txt file from dataset and preprocess the data.

4. **Part 3: The Seq2Seq Architecture**

   Task: Build the following two components using PyTorch:

   - The Encoder: A Linear layer that projects the 2048-dim cached vector into the hidden_size (e.g., 512).
   - The Decoder: An LSTM or GRU.
     - Input: Word Embeddings of the caption.
     - Initial Hidden State: The output of your Encoder.
     - Output: A Linear layer mapped to your vocab_size.

5. **Part 4: Training & Inference**

   - Loss Function: Use CrossEntropy Loss. Ensure you set ignore_index to your padding token's ID.
   - Optimization: Use the Adam optimizer.
   - Inference Function: Implement **Greedy Search** and **Beam Search** methods that generate words sequentially from an image feature until the <end> token is reached.

Deliverables

1. **Caption Examples:**

   - Display 5 random test images.
   - Show image, ground truth caption, and model-generated caption.

2. **Loss Curve:**
   - Plot training and validation loss over epochs.
3. **Quantitative Evaluation:**
   - BLEU-4 Score
   - Precision, Recall, F1-score on predicted captions (token-level or n-gram level)
   - Optionally: METEOR or ROUGE
4. **App Deployment:**
   - Create a Streamlit or Gradio app to demonstrate your model.